



Strapontin

Cometh Louis-Contracts

Security audit

26 November 2025

Summary

I. Introduction	4
1. About Strapontin	4
2. Risk Classification	4
a. Reporting	5
b. Table of Risk Classification	5
3. Disclaimer	6
4. Protocol Summary	6
5. Scope	6
II. Vulnerabilities	8
L-1 'emergencyWithdrawal' allows to remove pending payout's tokens	
I-1 An approved contract can send tokens directly to the recipient	
I-2 'TokenForwarder::withdraw' is not pausable	
I-3 No max-cap when updating the cooldown delay	
I-4 The function 'canExecutePayout' should return false when the contract does not have a sufficient amount of tokens	
I-5 'nonce' should be renamed to 'salt'	

I. Introduction

1. About Strapontin

Keeping decentralized projects secure is an essential part of building on the blockchain.

Strapontin is a Security Reasearcher specialized in Solidity. His objectives are to contribute to the ecosystem and support new projects by conducting security reviews.

You can see Strapontin's portfolio [here](#), and contact him on [X \(@OxStrapontin\)](#), or [Telegram \(strapontinOx\)](#).

2. Risk Classification

a. Reporting

Every point in the code is subject to internal discussions with the team. At this stage, a majority of the discovered issues have already been identified and documented.

Following the completion of the code review, analysis, and testing, we prepare a report. Each vulnerability is defined with a severity score:

Critical

A critical vulnerability represents a serious flaw that poses a major threat to the contract and its users. Such issues are often simple to exploit and may lead to fund losses or other severe outcomes. They demand prompt remediation to prevent potential damage.

High

A high vulnerability is a serious issue that can negatively impact the contract and its users, though to a lesser degree than a critical vulnerability. Such weaknesses are typically exploitable and could lead to fund losses or other adverse effects.

Medium

A medium vulnerability is an issue that might impact the contract or its users, but it is harder to exploit than high or critical vulnerabilities, or shows less impact than higher severities. While it may affect functionality or security, it can be resolved without major disruption.

Low

A low vulnerability is a minor issue that presents little to no risk to the contract or its users. Such issues are usually hard to exploit and often relate to cosmetic or technical details without affecting the contract's security or core functionality..

Informational

An informational finding is not a vulnerability but a recommendation or observation aimed at improving the contract. It may include best practice guidance, proposals for enhancing code clarity, or other non-critical improvements. Although not urgent, addressing these findings can enhance performance and lower the likelihood of future issues.

b. Table of Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	C	H	M
	Medium	H	M	L
	Low	M	L	L

3. Disclaimer

Our team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security review by the team is not an endorsement of the underlying business or product. The security review was time-boxed and the review of the code was solely on the security aspects of the implementation of the contracts. A security review does not prove that the code is bug-free. Gas and performance issues reported below (if any) will be grouped in the Informational category.

4. Protocol Summary

This system enables users to on-ramp from fiat to crypto by receiving EURC issued by Circle. The protocol maintains full custody over user wallets. The `TokenDistributor` contract allows the protocol to distribute tokens on-chain based on an operator's signature after the user invokes `requestPayout`. Once the defined cooldown period has elapsed, `executePayout` can be called to transfer the tokens to the end user's address.

For off-ramping, users can convert their stablecoins back to fiat through the `TokenForwarder` contract. Its `withdraw` function leverages Permit2 to transfer

tokens from the user to a trusted recipient.

Additionally, protocol administrators can pause the `TokenDistributor` contract in case of emergency, withdraw stuck tokens, and cancel pending payout requests. The cooldown period is also parameterizable to a non-zero value, and any change made to the cooldown period while pending payouts are active will immediately reflect on them.

5. Scope

```
/contracts/TokenDistributor.sol  
/contracts/TokenForwarder.sol
```

II. Vulnerabilities

L-1 'emergencyWithdrawal' allows to remove pending payout's tokens

Impact: Low

Description:

`emergencyWithdrawal` let the protocol admin transfer any ERC20 token to a recipient. This includes the transfer of token pending for distribution to users.

In case of a mistake, where the protocol team leaves the contract balance with less tokens than pending payouts, then users may experience a DoS when trying to execute their payouts.

Recommendation:

Consider adding a state variable that tracks the number of tokens pending for distribution. New requests should increase this variable, while request execution and cancellation should reduce it.

L-1 An approved contract can send tokens directly to the recipient

Impact: Informational

Description:

To off-ramp tokens, the `withdraw` function sets the permit with the Permit2 contract, transfer the funds to the TokenForwarder contract, and then transfer the funds again to the recipient. These 3 steps could be done in only one transaction, reducing the complexity in the token flow and gas consumption.

Recommendation:

Use `IPermit2.permitTransferFrom` to handle permit verification and transfer of tokens in one function, and update `from` and `to` to be directly the signer and final recipient.

I-2 'TokenForwarder::withdraw' is not pausable

Impact: Informational

Description:

In case of an emergency, all user functions can be paused in `TokenDistributor`. This protection has not been implemented in `TokenForwarder`. Since the recipient is an immutable address, it won't be possible to change it either if it gets compromised, and the contract will forever stay active if the team needs to shift to a new implementation for handling off-ramping users.

Recommendation:

Make the contract pausable to prevent it from remaining active if an issue occurs.

I-3 No max-cap when updating the cooldown delay

Impact: Informational

Description:

The function `setCooldownDelay` allows the admins to change the cooldown period between requesting a payout and executing it. The cooldown value can be set to any value between 1 second and without a maximum capacity.

Recommendation:

Consider adding a reasonable limit when changing the cooldown delay, so users can have a clear expectation of how long they may have to wait in a worst case scenario.

I-4 The function `canExecutePayout` should return false when the contract does not have a sufficient amount of tokens

Impact: Informational

Description:

`canExecutePayout` is a view functions that returns `True` if a request can be executed. However, if all conditions in the function are met but there are not enough tokens in the contract, `executePayout` will fail.

This forces the integration to perform additional checks for when this scenario might occur.

Recommendation:

Consider adding this check in the return condition:

```
function canExecutePayout(
    bytes32 payoutRequestHash
) external view returns (bool canExecute) {
    PayoutRequest memory request = payoutRequests[payoutRequestHash];
    return
        request.timestamp != 0 &&
        !request.executed &&
        !request.cancelled &&
        block.timestamp >= request.timestamp + cooldownDelay &&
+       token.balanceOf(address(this)) >= request.amount &&
        !paused();
}
```

I-5 `nonce` should be renamed to `salt`

Impact: Informational

Description:

A nonce is a value used once per operation, usually increasing by one after use, and tracked in the smart contract.

In `requestPayout`, the variable `nonce` is used as a random value added to a hash to reduce the chances of collision when the other parameters are unchanged. This description better fits

what's commonly called a salt.

Recommendation:

For readability, refactor nonce to be called salt.